

SG PE Syllabus Bot — Product & Technical Specification

Sep 24, 2026 · @amos

1. Overview

SG PE Syllabus Bot is a web app that answers questions on the 2024 MOE PE Syllabus and grades pupils' Fundamental Movement Skills (FMS) from video. It runs as a React single-page app on Vercel, with serverless functions proxying every AI call and Supabase for auth, data and storage.

Users

User	How they get in	What they do
Teacher	Google sign-in (Supabase Auth)	Syllabus Q&A, video grading, custom rubrics, PDF upload, plan lessons, run the class board, view pupil progress
Pupil	Scans the class QR (no account)	Pair check-in, peer coaching with AI feedback, Practice Station self-analysis
Visitor	No sign-in	Syllabus Q&A and analysis on Gemini only

Goals

- Answer syllabus questions from the syllabus text itself, not model memory.
- Grade the 10 FMS skills against the MOE checklist with frame-level evidence.
- Let a whole class record and review each other in pairs during a lesson, with capped AI cost.
- Keep pupil data private: no pupil logins, no public video URLs.

Non-goals

- No pupil accounts or long-term pupil logins.
- No native mobile app; the web app must work on phones and iPads.
- No server-side video processing; pose detection runs in the browser.

2. Architecture

The browser never holds an LLM key: every AI call goes to a Vercel function that adds the key, checks the caller and sets the model. Pose detection runs in the browser; Supabase holds auth, tables, vectors and video.

```

flowchart LR
    B[React SPA<br/>Vite + TS] -->|POST /api/gemini| G[Gemini 2.5 Flash]
    B -->|POST /api/claude| C[Claude Sonnet / Haiku]
    B -->|POST /api/rag-search| R[Gemini embeddings]
    R --> V[(Supabase pgvector)]
    B -->|supabase-js + RLS| S[(Supabase DB<br/>Auth, Storage)]
    B --> M[MediaPipe<br/>in browser]
    CR[Vercel cron<br/>23:55 SGT] --> S
  
```

The browser talks to Supabase directly under row-level security; the AI proxies are the only server code on the request path.

Tech stack

Layer	Choice
Frontend	React 19, TypeScript 5.8, Vite 7, Tailwind CSS
Hosting	Vercel: static SPA + Node serverless functions + one Edge function + 2 cron jobs
Auth, DB, storage	Supabase: Google OAuth, Postgres with RLS, pgvector, private Storage bucket
LLMs	Gemini 2.5 Flash (<code>@google/genai</code>), Claude Sonnet 4.6 and Haiku 4.5 (Anthropic Messages API via <code>fetch</code>)
Embeddings	<code>gemini-embedding-001</code> , truncated to 768 dimensions
Vision	<code>@mediapipe/tasks-vision</code> : Pose Landmarker (33 points) + Object Detector
Other libraries	<code>qrcode</code> + <code>html5-qrcode</code> (class QR), <code>idb</code> (offline store), <code>jspdf</code> + <code>html2canvas</code> (PDF reports), <code>pdf-parse</code> + <code>mammoth</code> (document ingest)

Code layout

Folder	Holds
<code>api/</code>	Serverless functions, each self-contained (no shared imports)
<code>src/App.tsx</code>	Chat app shell: sessions, message routing, video upload
<code>src/pages/</code>	<code>Dashboard</code> (pupil progress), <code>TeacherClassroomBoard</code> (live lesson)
<code>src/components/</code>	<code>chat</code> , <code>video</code> , <code>classroom</code> , <code>peer</code> , <code>admin</code> , <code>dashboard</code> , <code>layout</code>
<code>src/services/ai/</code>	Service registry, Gemini and Claude services, peer coaching, pupil AI headers
<code>src/services/vision/</code>	MediaPipe wrapper and biomechanics maths
<code>src/data/</code>	Syllabus text, router, FMS and gymnastics checklists, few-shot examples
<code>supabase_*.sql</code>	Schema, RLS policies and RPC functions, applied by hand in the SQL editor

3. Roles and access control

The server decides who may spend money: Claude answers only a signed-in teacher or a pupil holding today's lesson pass, and the server, never the request body, picks the model and output cap.

Who may call which model

Caller	Proof sent	<code>/api/gemini</code>	<code>/api/claude</code>	Model chosen by server	Output cap
Visitor	nothing	Yes	No (401, "sign in")	Gemini 2.5 Flash	2,000 tokens (default 1,200)
Teacher	<code>Authorization: Bearer <Supabase token></code>	Yes	Yes	Sonnet 4.6 or Haiku 4.5 as requested, else Haiku	4,000 tokens

Caller	Proof sent	/api/gemini	/api/claude	Model chosen by server	Output cap
Pupil, peer feedback	X-Lesson-Id , X-Lesson-Pass , X-Pair-Number , X-Performer: pair , X-Ai-Purpose: peer_feedback	Yes	Yes, 12 calls per pair per lesson	Haiku	1,024 tokens
Pupil, Practice Station	same headers, performer apple or banana , purpose analysis or question	Yes	Yes, 5 calls per pupil per lesson	First analysis Sonnet, rest Haiku	4,000 tokens

- Gemini allows only the Google Search tool; any other tool in the body is dropped.
- Claude requests are limited to 40 messages and 32 images; Gemini history to 60 turns.
- CORS allows only `ALLOWED_ORIGIN` plus localhost.
- If Anthropic fails, the pupil's call is refunded via `pupil_ai_refund()` , including their Sonnet turn.
- In the UI: signed out → Gemini only, Claude greyed out; a pupil in the Practice Station → Claude.

The lesson pass

Each lesson has a secret `pupil_pass` (random UUID without dashes), carried in the class QR with the lesson id. A pupil action is accepted only when all three hold:

1. The lesson exists (a teacher planned it).
2. `lesson_date` equals today in Singapore time (`sg_today()`).
3. The pass matches.

The lesson's own `teacher_id` is used for every pupil write; a teacher id sent by the device is ignored. The owning teacher, when signed in, may write to their own lesson on any day without the pass (for testing on an iPad).

Pupil data lockdown

- Pupils are never signed in and have no direct table access to `pair_sessions` or `pair_submissions` . They go through `SECURITY DEFINER` functions: `pupil_lesson_status` , `pupil_claimed_pairs` , `pupil_check_in` , `pupil_save_submission` , `pupil_get_submission` .
- Teachers read, update and delete only rows where `teacher_id = auth.uid()` .
- A **claim token** (random id per device per lesson, kept in `localStorage`) stops a second group taking a pair number already in use; the function returns `claimed` .

- A trigger stops pupil writes from erasing teacher feedback, stars or AI analysis. Re-submitting reviewed work sets status to `resubmitted`.
- The `student-videos` bucket is private. Pupils may upload only to `<teacher_id>/pair_submissions/<lesson_id>/...` for a lesson dated today. Teachers play clips through short-lived signed URLs.
- No `using (true)` policies on pupil tables.
- The budget functions are executable only by the service role, so browsers cannot call them.

4. Feature: Syllabus Q&A

A teacher asks a question in chat and gets a short answer drawn from the 2024 MOE PE Syllabus text, steered to the right level and learning area through clickable choice chips.

Context the model receives

1. **System instruction:** the answer rules below, plus the checklist for the active FMS skill if one is selected.
2. **Syllabus prefix:** a user/assistant message pair holding the syllabus (introduction, all level learning outcomes, pedagogy, assessment, glossary) built by `getSyllabusContextMessage()`. On Claude it carries `cache_control: ephemeral` for prompt caching.
3. **RAG excerpts:** for any message over 3 characters, `/api/rag-search` returns up to 4 chunks from teacher-uploaded PDFs (similarity threshold 0.3, 30 s timeout). Failure is silent.
4. **Teacher memory** (Claude, signed in, text only): the last 3 days of nightly summaries from `/api/get-memory`, prepended to the system instruction.
5. **Recent history:** capped to a short-term window of recent messages.

The full syllabus text is about 455 KB, stored in `src/data/syllabusData.ts`. A keyword router (`getSyllabusContext`, 80 KB cap per section) exists as a lighter alternative for models without prompt caching.

Answer rules

Rule	Trigger	Required response
1. Brevity	Every text answer	At most 4 sentences or 5 bullets, one topic

Rule	Trigger	Required response
2A. Vague	No level, no learning area	One short line, then 3–4 level + area chips
2B. Semi-specific	Level + area, no sub-category	One line, then sub-category chips (must use chips)
2C. Specific	Sub-category known or chip clicked	Full numbered list of outcomes for that sub-category, then 3 follow-up chips
3. Skill criteria	"critical elements", "checklist", "how to perform"	Performance criteria from skill data plus a reference image tag; skip tiers
4. Media uploaded	Video or image	Top 3 likely skills as chips
Out of syllabus	Topic not in syllabus	Say so; one sentence from Google Search grounding (Gemini)

Sub-category chips per learning area:

Area	Chips
Games & Sports	Sending & Receiving, Sending, Propelling, Concepts & Safety Practices
Athletics	Running, Jumping, Throwing, Combined Events
Dance	Locomotor, Non-Locomotor, Manipulative Skills, Dance Phrases
Gymnastics	Travelling, Jumping & Climbing, Balancing, Rotating, Mounting, Dismounting & Vaulting
Swimming	Water Safety, Floating & Gliding, Strokes, Turns & Starts
Outdoor Education	Orienteering, Camping & Survival, Environmental Awareness

Response tags the frontend parses

Tag	Rendered as
<code>[[SKILL_CHOICES: A, B, C]]</code>	Single-select chip buttons; clicking one sends it as the next message
<code>[[MULTI_SKILL_CHOICES: A, B, C]]</code>	Multi-select chips with a confirm button (e.g. skills in a gymnastics sequence)
<code>[[DISPLAY_REFERENCE: Skill Name]]</code>	The skill's reference image from <code>public/assets/reference_images/</code>

Tags are stripped from the visible markdown. Every reply must contain visible text before a chip tag.

Chat UI requirements

- Sessions list in a sidebar: create, rename, delete, switch.
- Composer: text, voice input (Web Speech API), file upload (video, image), camera recording, optional "Target Skill" field.
- Model toggle: Gemini or Claude; Claude disabled when signed out.
- Markdown rendering with tables.
- Offline: sessions readable from local storage without a network.

5. Feature: Motion analysis

A teacher uploads or records a clip; the browser extracts pose landmarks, computes a biomechanics report, and the LLM first identifies the skill, then grades each checklist criterion with frame evidence and a proficiency level.

```

flowchart LR
    A[Upload or record clip] --> B[Trim start/end]
    B --> C[MediaPipe pose +<br/>ball detection]
    C --> D[Biomechanics report]
    D --> E{Target skill given?}
    E -->|No| F[Phase 1: 4 skill<br/>guesses as chips]
    F --> G[Phase 2: grade]
    E -->|Yes| G
    G --> H[Criteria + proficiency<br/>+ skeleton replay]
```

Phase 1 is skipped whenever the skill is already known: typed in the Target Skill field, or fixed by the lesson.

Skill modes

Mode	Skills	Source
FMS	Underhand Throw, Underhand Roll, Overhand Throw, Kick, Dribble with hands, Dribble with feet, Chest Pass, Catch above waist, Bounce pass, Bounce (10)	<code>fundamentalMovementSkillsData.ts</code>
Gymnastics	Locomotor (8), jump shapes (5), upright balances (4), inverted balances (4), partner balances (2), rolls (6), spinning and turning (3), vaults (3): 35 total	<code>gymnasticsSkillsData.ts</code>

Each skill has a performance-criteria checklist, a reference image and few-shot grading examples (`skillExamples.ts`). For a gymnastics clip the app first asks: individual skill or sequence? A sequence uses multi-select chips.

Pose extraction (browser)

- MediaPipe Pose Landmarker `pose_landmarker_full` (float16), CPU delegate, 1 pose, 33 landmarks per frame, plus world landmarks.
- MediaPipe Object Detector tracks the ball; each detection is filtered by proximity to the hands and by movement, and labelled valid or ignored.
- Frames are sampled from the trimmed range; selected frames are sent as images with the report (Claude limit 32 images).
- Replay player draws the skeleton and a ball "comet tail" on a canvas over the video.
- A SHA-256 hash of the video file keys a cache so the same clip is not analysed twice.

Biomechanics report fields

Field	Computed from	Used for
Camera orientation	Shoulder span ÷ torso height, plus left/right visibility asymmetry	Whether left/right labels can be trusted; if side-on, report says "lead foot" instead
Throwing arm	Wrist (15 or 16) with more total travel	Which arm throws; swap-invariant

Field	Computed from	Used for
Arm trajectory	Highest wrist point vs nose Y and hip Y	Primary classifier: overhead, low swing or mid-level
Wind-up	Wrist ever below hip Y	Underhand vs overhand backswing
Lead foot	Ankle furthest along throw direction near release, within the setup-to-release window	Step toward target
Coordination	Throwing wrist index vs lead ankle index	Same side = ipsilateral error
Stance	Ankle gap vs shoulder width	Feet shoulder-width apart
Knee bend	Minimum knee angle, with frame number	Knees slightly bent
Step detection	Max ankle distance ÷ initial > 1.2	Step taken

Y runs 0 (top) to 1 (bottom). In gymnastics mode arm trajectory is ignored; the report focuses on flight phase, landing knee bend and rhythm.

Grading rules (Phase 2)

- Each criterion marked  met,  not met, or  unclear, with frame numbers as evidence.
- Proficiency: **Beginning** under 50% of criteria; **Developing** 50–80%; **Competent** all criteria; **Excellent** all criteria plus exceptional quality.
- Smoothness rule: all  but robotic, segmented movement → Developing.
- A teacher's custom rubric for the skill (`customRubrics[skillName]` , built in the Rubric Builder) replaces the standard checklist entirely.
- Output includes pupil-friendly feedback and next-step cues. Signed-in teachers can save the result to a named pupil's record (`skill_analyses`) and export a PDF report.

6. Feature: Lessons, class QR and peer coaching

A teacher plans a lesson, projects its QR code, and pupils in pairs ("Apple" and "Banana") scan it on a shared iPad or phone, film each other, tick peer cues, and send the work to the teacher's live board with AI feedback.

sequenceDiagram

```

participant T as Teacher board
participant P as Pupil device
participant S as Supabase
participant A as /api/claude
T->>S: Create lesson (id, pass, date)
T->>P: Project QR {lessonId, pass, teacherId, skill, pairCount}
P->>S: pupil_lesson_status
P->>S: pupil_check_in (pair no., photo, claim token)
P->>P: Apple films Banana, ticks cues, then swap
P->>S: Upload clips + pupil_save_submission
P->>A: peer_feedback (Haiku)
T->>S: Review, star, approve or redo
P->>S: pupil_get_submission (teacher comment)

```

Lesson planner (teacher)

Field	Rule
Date	Defaults to today; pupil access works only on this date (Singapore time)
Class name	Required
Level	P1-P6, optional
Objective	Free text, optional
Skill area	FMS or Gymnastics
Skill	From the area's skill list
Pairs	1-30, default 15

Every lesson gets a new id; ids are never reused because check-ins, submissions and storage folders are keyed by them. Which lesson is on the projector is stored per device.

Pupil flow

1. **Scan QR** with the camera (html5-qrcode), or snap a photo of it when the camera is blocked (iOS over HTTP). Reject non-lesson codes and codes for another day with plain messages.
2. **Pair check-in:** choose a free pair number (taken ones greyed out), take a pair photo with a countdown. Spoken prompts guide each step.

3. **Peer coaching**, seven steps: Apple intro → Apple records Banana → Apple reviews replay and ticks cues → swap → Banana records Apple → Banana reviews → done. Cues come from `peerSyllabusCues.ts` (core and full lists per skill). A skeleton replay supports the review.
4. **Send**: clips upload to the private bucket; the record queues offline first (IndexedDB) and syncs when online.
5. **Automatic AI feedback**: 4 Haiku calls over 4 frames each (max 640 px). Returns pupil feedback for each partner and a teacher report with proficiency per pupil, recommendations, and disagreements between peer ticks and AI judgement.
6. **Help beacon**: a button that flags the pair as needing help on the teacher board.

Practice Station (pupil, after sending)

- A chat screen per pair with buttons: Analyse Apple, Analyse Banana (full rubric grading), two quick questions, Send to Teacher.
- Always uses Claude through the lesson pass; 5 calls per pupil, first analysis on Sonnet.
- Each analysis is filed under the right pupil and keeps its video when later feedback arrives.
- Polls the pair's submission for a teacher comment and shows it in the chat; a pair can resume its chat later in the lesson.
- Layout fits phones and iPads in portrait.

Teacher classroom board

- Projector view: lesson title, large QR, pair grid showing check-in photo, status and help flags.
- Submissions list with count of unreviewed items (`pending_sync` or `resubmitted`).
- Per submission: play both clips (signed URLs), see peer cues vs AI report, write feedback, **Approve** (optional star) or **Needs redo**. Default comments: "Well done pair!" and "Please try again with partner."
- Clear a pair's check-in; delete submissions and their videos.
- Status values: `pending_sync` , `synced` , `approved` , `needs_redo` , `resubmitted` .

Progress dashboard (teacher)

- Pupil list (`students`) with each pupil's saved skill analyses and proficiency over time.
- Nightly cron summarises new analyses per pupil per skill into a progress note.

7. Data model

Supabase Postgres is the source of truth for signed-in teachers; the browser keeps a local copy (localStorage for chats, IndexedDB for classroom work) so the UI is instant and works offline, then merges with the cloud on load.

Tables

Table	Key columns	Who can access
teacher_profiles	id (= auth user), email, name, avatar_url, custom_rubrics jsonb {skill: rubric}	Own row
chat_sessions	id, user_id, title, messages jsonb, archived, timestamps	Own rows
user_memory_archive	user_id, summary_date, summary_text	Own rows (read); cron writes
students	teacher_id, index_number (unique per teacher), name, class, progress_summary jsonb {skill: text}	Own pupils
skill_analyses	student_id, teacher_id, skill_name, video_hash, video_url, proficiency_level, analysis_text, pose_data, model_id, token_usage, summarised	Own rows
lessons	id text, teacher_id, lesson_date, class_name, level, objective, skill_area (FMS or Gymnastics), skill_name, pair_count 1-30, pupil_pass	Owner only; pupils never read it
pair_sessions	id = <lesson>-p<n>, lesson_id, pair_number, skill_name, teacher_id, pair_photo, needs_help, claim_token	Teacher: own rows. Pupil: RPC only

Table	Key columns	Who can access
<code>pair_submissions</code>	<code>id</code> , <code>lesson_id</code> , <code>pair_number</code> , <code>skill_name</code> , <code>teacher_id</code> , <code>pair_photo</code> , <code>apple_video_url</code> , <code>banana_video_url</code> , <code>apple_cues</code> , <code>banana_cues</code> , <code>ai_student_feedback</code> , <code>ai_teacher_report</code> , <code>ai_chat_analysis</code> , <code>status</code> , <code>teacher_feedback</code> , <code>teacher_star</code> , <code>claim_token</code>	Teacher: own rows. Pupil: RPC only
<code>ai_usage</code>	PK (<code>lesson_id</code> , <code>pair_number</code> , <code>performer</code>), <code>calls</code> , <code>sonnet_used</code>	Teacher reads own lessons; server writes
<code>documents</code>	<code>id</code> , <code>title</code>	Upload pipeline
<code>document_chunks</code>	<code>document_id</code> , <code>content</code> , <code>embedding vector(768)</code> , HNSW cosine index	Search via <code>match_document_chunks</code>

Functions

Function	Caller	Returns
<code>sg_today()</code>	internal	Today's date in Asia/Singapore
<code>pupil_lesson_status(lesson, pass)</code>	pupil	<code>ok</code> , <code>not_today</code> , <code>invalid</code>
<code>pupil_claimed_pairs(lesson, pass)</code>	pupil, teacher	Taken pair numbers
<code>pupil_check_in(...)</code>	pupil, teacher	<code>ok</code> , <code>claimed</code> , <code>invalid_lesson</code>
<code>pupil_save_submission(jsonb)</code>	pupil, teacher	<code>ok</code> , <code>claimed</code> , <code>invalid_lesson</code>
<code>pupil_get_submission(id, claim_token)</code>	pupil	The pair's own submission
<code>pupil_can_upload(path)</code>	storage policy	Whether a pupil upload path is allowed

Function	Caller	Returns
<code>pupil_ai_use(...)</code> / <code>pupil_ai_refund(...)</code>	service role only	<code>{ok, model}</code> or <code>{ok:false, reason}</code>
<code>match_document_chunks(embedding, threshold, count)</code>	RAG endpoint	Top matching chunks
Triggers: <code>protect_teacher_columns</code> , <code>enforce_pair_claim</code> , <code>enforce_submission_claim</code>	automatic	Guard teacher fields and pair claims

Storage

- Bucket `student-videos`, private. Path: `<teacher_id>/pair_submissions/<lesson_id>/<file>`.
- Pupils insert only to a path for their teacher's lesson dated today; teachers read and delete only under their own `teacher_id` folder; playback through signed URLs.

Browser storage

Store	Holds
localStorage	Chat sessions (without video data), selected model, projector lesson per device, lesson pass per lesson, pair claim token
IndexedDB <code>idb</code>	<code>pair_session</code> (current pair), <code>submissions</code> (offline queue, indexed by status and lesson), <code>lesson_cache</code>
In-memory map	Video base64 by <code>videoId</code> , re-attached after a cloud sync strips binaries; lost on refresh

Merge rule: load local first, then fetch cloud; for signed-in teachers cloud wins. The session fetch depends on `user.id`, not the `user` object, so token refreshes don't re-fetch and drop in-memory video.

8. API contracts

Seven endpoints, each a self-contained file under `api/` with no shared imports. All API endpoints return `{ error: string }` with a plain-language message on refusal.

Endpoint	Method	Auth	Request	Response
<code>/api/gemini</code>	POST	None (CORS-limited)	<code>{ history, message, systemInstruction, tools?, maxOutputTokens? }</code>	<code>{ text, tokenUsage, groundingChunks, finishReason }</code>
<code>/api/claude</code>	POST	Teacher bearer token or pupil lesson headers	<code>{ messages, system?, model?, max_tokens?, temperature? }</code> (Anthropic Messages shape)	<code>{ text, tokenUsage }</code>
<code>/api/rag-search</code>	POST	None	<code>{ query }</code>	<code>{ context }</code> (joined excerpts, empty when nothing matches)
<code>/api/upload-pdf</code>	POST multipart, field <code>file</code>	Teacher bearer token	PDF file	Document id, chunk count, errors
<code>/api/get-memory</code>	GET (Edge)	Teacher bearer token	none	<code>{ summaries: [{ summary_date, summary_text }] }</code> , last 3 days; empty if no token
<code>/api/summarise-progress</code>	GET (cron)	<code>Bearer CRON_SECRET</code>	none	Counts processed
<code>/api/daily-chat-archive</code>	GET (cron)	<code>Bearer CRON_SECRET</code>	none	Counts archived

Status codes for `/api/claude`

Code	Meaning	Message shown
401	No proof, or expired sign-in	"Sign in to use Claude. Gemini is available without signing in."
403	Pass wrong or lesson not today	"This lesson's QR code isn't open today..."
429	Pupil budget used up	"You've used all your AI feedback for this lesson..."
400	Too many messages or images, unknown purpose	"Invalid request..."

Code	Meaning	Message shown
502 / 503	Anthropic failed (503 when Anthropic rate-limits); pupil call refunded	Upstream error text

Upload pipeline

1. Verify teacher token.
2. Parse the PDF (`pdf-parse`); reject a PDF with no text (422).
3. Chunk with running level + subject headers prepended to each chunk, so each retrieved chunk says where it came from; drop chunks under 50 characters.
4. Embed each chunk with `gemini-embedding-001` , truncate to 768 dimensions, insert into `document_chunks` .

Nightly jobs (23:55 Singapore, `55 15 * * * UTC`)

- **summarise-progress**: groups unsummarised `skill_analyses` by pupil and skill, asks Haiku (256 tokens) to update the pupil's `progress_summary[skill]` , marks rows summarised.
- **daily-chat-archive**: for each teacher, sends the previous Singapore day's chats to Haiku, stores a markdown summary in `user_memory_archive` , marks sessions archived. Never mixes teachers.
- Both use the service role key and refuse to run without `CRON_SECRET` .

9. Non-functional requirements

The rebuild must keep pupil data private, keep AI spend bounded per lesson, and run on a shared school iPad in a hall with patchy Wi-Fi.

Security and privacy

- No API key for an LLM in the browser bundle. Only `VITE_SUPABASE_URL` , `VITE_SUPABASE_ANON_KEY` are public.
- Service role key used only by cron jobs and the pupil budget check; never `VITE_` -prefixed.
- RLS on every table; no `using (true)` policies for pupil data.
- No pupil accounts, names only as teacher-entered records; pupil devices identified by lesson pass + claim token.
- Video never served by public URL.

- Teacher memory endpoint identifies the teacher from the verified token, never a supplied id.

Cost limits

Limit	Value
Pupil peer feedback	12 Haiku calls per pair per lesson
Pupil Practice Station	5 calls per pupil per lesson, 1 on Sonnet
Teacher Claude output	4,000 tokens per call
Gemini output	2,000 tokens max, 1,200 default
Claude request size	40 messages, 32 images
Syllabus prefix	Prompt-cached on Claude

Devices and performance

- Works on iPad Safari and phone browsers in portrait; teacher board also on a projector at desktop width.
- Camera needs HTTPS (dev server uses basic SSL); fallback to photo of the QR.
- Pose detection runs on CPU in the browser; analysis caches by video hash.
- Serverless function timeout 60 s for Claude.
- Offline: submissions queue in IndexedDB and sync when back online; chats readable offline.
- Light and dark themes.

Environment variables

Variable	Where	Purpose
VITE_SUPABASE_URL , VITE_SUPABASE_ANON_KEY	Browser	Supabase client
GEMINI_API_KEY	Server	Gemini chat, embeddings
ANTHROPIC_API_KEY	Server	Claude
SUPABASE_URL , SUPABASE_ANON_KEY	Server	Token checks, RAG search
SUPABASE_SERVICE_ROLE_KEY	Server	Cron jobs, pupil budget

Variable	Where	Purpose
<code>CRON_SECRET</code>	Server	Authorises cron calls
<code>ALLOWED_ORIGIN</code>	Server	CORS origin, e.g. the production URL

Local development

- `yarn dev` serves at `https://localhost:5173` on the LAN so iPads can test.
- A Vite `claude-dev-proxy` stands in for `/api/claude` locally; access checks apply only on Vercel.
- `yarn typecheck` and `yarn typecheck:api` are the only automated checks today.

10. Rebuild plan

Build in six milestones, each shippable on its own; security and pupil budgets land before any pupil-facing feature, not after.

#	Milestone	Done when
1	Foundation: Vite + React + TS, Supabase project, Google sign-in, schema and RLS from section 7, Vercel deploy	A teacher signs in; a second account cannot read the first's rows
2	Syllabus Q&A: <code>/api/gemini</code> , <code>/api/claude</code> (teacher path), syllabus prefix, answer rules, chip and reference-image tags, sessions sidebar, cloud sync	"What are the learning outcomes?" returns level chips; clicking through reaches a full numbered list
3	RAG and memory: PDF upload, embeddings, <code>/api/rag-search</code> , nightly chat archive, <code>/api/get-memory</code>	An uploaded PDF's content appears in an answer; yesterday's chat summary is in today's context
4	Motion analysis: MediaPipe, trim UI, camera recorder, biomechanics report, Phase 1 and 2, skeleton replay, custom rubrics, save to pupil, PDF export	A known overhand throw clip is identified and graded with frame evidence; a side-on clip reports "lead foot" not left/right
5	Lessons and classroom: planner, lesson pass, QR, pupil RPCs, private bucket, check-in, peer coaching, offline queue, teacher board	Two groups can't claim the same pair; a pupil can't write to yesterday's lesson; teacher approves a submission

#	Milestone	Done when
6	Pupil AI: lesson headers, <code>pupil_ai_use</code> budgets and refunds, peer feedback, Practice Station, progress dashboard and nightly summary	The 13th peer-feedback call for a pair returns 429; a failed Anthropic call does not use up a turn

Acceptance checks across all milestones

- ☐ No LLM key appears in the built JavaScript.
- ☐ Every table has RLS on; no open policies on pupil tables.
- ☐ Pupil screens fit an iPad and a phone in portrait without horizontal scroll.
- ☐ Signed-out users see Claude greyed out and can still chat on Gemini.
- ☐ Token refresh does not reload sessions or drop in-memory video.

Open questions

- Keep both Gemini and Claude, or standardise on one provider for the rebuild?
- Keep the whole-syllabus prompt prefix, or switch to the keyword router or RAG-only to cut tokens?
- Add automated tests (none exist today): at least RLS policy tests and the biomechanics maths.